

LLM Training Optimization Assessment

Jeremy Zhang*

The University of Melbourne

January 30, 2026

Abstract

We optimized a small GPT-2 style language model trained from scratch on a fixed Hacker News headlines dataset under strict constraints. Starting from the provided baseline (validation loss = 1.754), we improved the validation loss to **1.108043355** through extensive experiments on an NVIDIA RTX 4090 by tuning training hyperparameters, refining the model architecture, and improving the data pipeline. Because this is a short report, we summarize only the key rationale and changes. In addition, we conducted sample-output comparison experiments between the baseline model and the best optimized model. The optimized code and Git log are available on GitHub (<https://github.com/sqzhang-jeremy/mainrun>).

1 Problem & Constraints

Goal: minimize validation loss within 7 epochs on the provided dataset, a Hacker News headline dataset (100,000 Hacker News titles, 90% for train and 10% validation).

Fixed: epochs, random seed, dataset, validation fraction, and the `evaluate()` function.

Allowed: model architecture, initialization, tokenization, hyperparameters, optimizer, scheduler, training loop, etc.

Not Allowed: pre-trained weights or augment the training data.

2 Baseline

The starter configuration achieves validation loss = 1.754 after 7 epochs.

3 Milestones

To make the iteration path clear to reviewers, we summarize the major milestones below:

1. **Local setup (MacBook M2 Pro, 16 GB RAM):** initial CPU-only runs failed with an out-of-memory error.
2. **Remote setup (RTX 4090):** moved training to a GPU server and achieved stable end-to-end runs.
3. **Iteration 1 (general ML optimization):** tuned core hyperparameters and adjusted the model architecture for better convergence.
4. **Iteration 2 (task-specific refinement):**
 - audited the dataset and its distribution;
 - revisited the data pipeline and tokenization choices;
 - explored architecture changes (e.g., sparse attention).

*Email: shiquanz@student.unimelb.edu.au; sqzhang.jeremy@gamil.com

5. **Finalization:** tested the best optimized model with some prompts, reorganized, analyzed experiment logs, plotted loss curves, and consolidated findings into this report.

4 Approach

4.1 High-level Overview

Training a small GPT-2 style decoder-only transformer model is similar to classical deep learning models, with the key differences being the autoregressive architecture and data tokenization. In general, from an optimization perspective, we can group the work into five buckets: (1) data pipeline and tokenization, (2) model architecture, (3) optimization and training hyperparameters, (4) regularization and generalization, and (5) evaluation hygiene. Given the constraints and scope of this challenge, we focus on the first three.

Following the theoretical framework above, I iterated in a simple, chronological loop: establish a stable baseline, optimize core hyperparameters and architecture, then revisit the data pipeline and tokenization once training dynamics plateaued.

After reviewing the starter code, I found that the initial model/training configuration was fairly plain and that headlines were concatenated into a single long stream, leaving significant room for improvement. To manage risk and maximize return on engineering time, I first focused on “safe” changes—standard hyperparameter sweeps (learning rate, batch size, and regularization) followed by modest architecture adjustments—to confirm that the training loop behaved as expected and that validation loss improved reliably.

Once progress slowed, I shifted attention to the data pipeline and tokenization. This analysis revealed several opportunities to reduce waste and improve token efficiency. Following this direction, I also explored sparse attention to reduce unnecessary full causal self-attention and emphasize local context, which better matches the short, headline-like structure of the dataset.

In retrospect, I should have analyzed the data earlier. There is still room for further gains (e.g., broader hyperparameter sweeps), but I prioritized consolidating results and writing this report. Despite a few missteps, these iterations helped me evaluate a wide range of optimization techniques and strengthen my understanding of practical LLM training.

4.2 Data Pipeline and Tokenization

4.2.1 Exploratory Data Analysis

To better understand the data distribution, we explored several dataset statistics. Figure 1 shows the title length distribution in the dataset.

Across more than 4M titles, the title length ranges from 0 to 198, while the word count ranges from 0 to 36 (median word count: 8). For example: “Ask HN: cloud data storage cheaper than AWS S3?”. Overall, the EDA suggests that most titles are short and that the dataset contains substantial noise.

4.2.2 Tokenization

Based on the exploratory data analysis, we identified several inefficiencies in the baseline tokenization pipeline and conducted controlled ablation experiments against our best model.

- **Lowercase normalization:** the baseline treated “Google”, “google”, and “GOOGLE” as distinct sequences. Lowercasing collapses case variants into canonical forms, yielding denser training signal per vocabulary slot. This was the single most impactful tokenizer change.
- **Whitespace pre-tokenizer:** replaced ByteLevel pre-tokenization (designed for multilingual/binary input) with whitespace splitting, constraining BPE merges to within word boundaries for more meaningful subwords.
- **Fix EOS token handling:** the baseline tokenized the entire concatenated corpus including literal <eos> strings, allowing BPE to merge delimiters with adjacent characters. We switched to tokenizing each title independently and appending the EOS token ID directly.

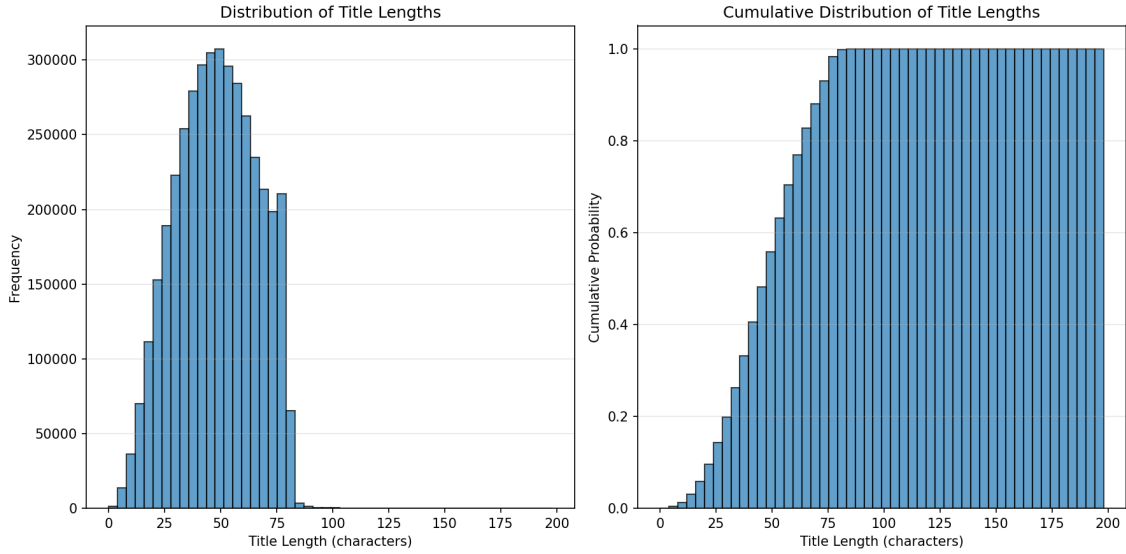


Figure 1: Title length distribution for the Hacker News headlines dataset.

- **Minimum frequency filtering:** setting `min_frequency=2` prunes singleton subwords, freeing vocabulary capacity for better-attested tokens.
- **Domain special tokens:** registered “Show HN:”, “Ask HN:”, and “Launch HN:” as atomic special tokens to prevent BPE fragmentation of these structured prefixes.
- **Vocabulary size sweep:** tested 8k/12k/16k/24k. With lowercase and min-frequency pruning, 24k performed best—the extra slots captured genuinely useful complete words rather than noise, shortening sequences.
- **Title-aware batching (unsuccessful):** stream-based batching creates context windows spanning unrelated titles. We implemented per-title batching with padding and `ignore_index` masking to eliminate cross-title noise. This did not improve validation loss, likely because the evaluation function uses the same stream format: it walks sequentially through a concatenated token stream where titles are separated only by EOS tokens, with context windows routinely spanning multiple titles.

Overall, the ablations suggest that most tokenization changes offered only marginal gains once the training setup was stable, while lowercase normalization consistently improved optimization by collapsing redundant surface forms and increasing effective token frequency. In practice, we therefore treated tokenization as a “signal-to-noise” and “token-efficiency” lever: keep changes that increase consistent statistics (e.g., normalization and clean EOS handling), and avoid changes that alter the evaluation distribution.

Based on this perspective, we adopted a lightweight tokenization bundle (lowercasing + improved BPE configuration/EOS handling + a tuned vocabulary size) and carried it forward as the final tokenization setting before moving on to further architecture-level changes (e.g., sparse attention).

4.3 Optimization and Training Hyperparameters

Because training was limited to 7 epochs, conventional “core” hyperparameters had an outsized impact on both convergence speed and final validation loss. We therefore started with changes that are broadly reliable in transformer training and easy to validate with controlled ablations.

- **Optimizer (SGD $\rightarrow$ AdamW):** replaced SGD with AdamW and tuned the β parameters to match typical transformer defaults.
- **Learning rate and schedule:** swept the learning rate (roughly 6×10^{-3} down to 3×10^{-4}) and added a warmup + cosine decay schedule (the baseline used cosine annealing). This reduced early instability while preserving strong progress later in training.

- **Weight decay and parameter groups:** ran multiple weight-decay settings (e.g., $0.1 \rightarrow 0.0 \rightarrow 0.1$) and fixed the parameter grouping:
 - apply weight decay to matrix weights (e.g., **Linear** and **Embedding** weights);
 - do *not* apply weight decay to biases or normalization parameters.
- **Dropout:** evaluated dropout rates 0.1, 0.05, and 0.0 to balance regularization against optimization speed.
- **Batch sampling determinism:** fixed sources of batch sampling randomness to reduce run-to-run variance and make small changes easier to attribute.

We also tried several ideas that did not help in this setting, including more aggressive AdamW β tuning and exponential moving average (EMA) of weights. EMA can be effective for longer or noisier training runs, but under a strict 7-epoch budget it can lag behind the rapidly improving online weights and underperform.

Overall, this phase established a stable optimization baseline: fast convergence (via AdamW + warmup/cosine), sensible regularization (weight decay + dropout), and lower run-to-run variance. With training dynamics under control, subsequent gains were more likely to come from architectural choices.

4.4 Model Architecture

4.4.1 Block Size and Batch Size Tuning

Because Hacker News headlines are short, a smaller context window is often sufficient. We therefore tuned the context length (**block_size**) and found that **block_size=64** captured most local structure while reducing wasted attention on padding/unrelated history.

Reducing **block_size** lowers the quadratic attention cost ($\mathcal{O}(L^2)$ in sequence length), which can be “re-invested” in either (1) more optimization steps (e.g., smaller batch size or less gradient accumulation), or (2) larger models (more layers / wider hidden size) at similar wall-clock time. In later iterations, we kept the effective tokens-per-step approximately constant while using the improved throughput to run more steps within the fixed 7-epoch budget.

4.4.2 GPT-2 Style Residual Projection Scaling

We adopted GPT-2 style residual projection scaling by reducing the initialization scale of (1) the attention output projection and (2) the MLP output projection. Concretely, these residual-producing projections were initialized with a smaller standard deviation, which helps keep residual updates well-behaved early in training and improves stability for deeper stacks.

4.4.3 Model Size Sweep

The training corpus contains only about 7.7M tokens in total, so a moderately large model (e.g., ~27M parameters) can be under-trained under a fixed 7-epoch regime. In this setting, a slightly smaller model can sometimes achieve lower validation loss by matching capacity to data and converging faster.

We conducted ablations that varied depth and width across a small grid, including:

- depth sweep: **n_layer** $\in \{4, 6, 8, 10, 12\}$;
- width sweep: **d_model** $\in \{384, 512\}$;
- deeper-narrower variants at fixed head dimension (holding attention head size constant while trading width for depth).

These experiments helped identify a “sweet spot” where the model was large enough to model the headline distribution but small enough to fit the available tokens and optimization budget.

4.4.4 MLP Ratio and SwiGLU

We also evaluated the feed-forward expansion (MLP ratio) by sweeping values in the range 3.0–6.0, trading off compute allocation between attention and the MLP.

Finally, we tested a gated SwiGLU MLP as a drop-in replacement for the standard GELU MLP while keeping the total parameter count approximately constant. Although SwiGLU often improves quality-per-parameter in decoder-only models, it did not improve validation loss in our constrained setup, so we kept the simpler GELU MLP for the final configuration.

4.4.5 Sparse Attention

The exploratory data analysis suggests that most headlines are short, often on the order of only a handful of words. Our tokenization experiments already addressed this from a token-efficiency perspective; here we approach the same observation from a *spatial* perspective.

Full causal self-attention allows every token to attend to all previous tokens, with $\mathcal{O}(T^2)$ compute in sequence length. In a concatenated headline stream, however, the most useful predictive signal for the next token is usually local: it tends to depend on nearby words within the same title, rather than tokens far back in the context that likely belong to a different headline.

To better match this structure, we experimented with sparse attention that restricts attention to a local window [1]. This introduces an inductive bias that prioritizes local context and reduces cross-title noise, instead of spending capacity on distant, largely irrelevant dependencies.

We also ran controlled ablations to disentangle what drives the gains in our sparse-attention design. In particular, we studied (1) whether improvements come from the sparse mask itself versus simply seeing longer context, and (2) the effect of adding a compression component via a bottleneck projection.

5 Git Commit Overview

To make the engineering iteration path auditable, we group the main Git commits into the themes below:

- **Infrastructure:** GPU setup, CLAUDE.md.
- **Learning rate & scheduler:** warmup + cosine decay.
- **Weight decay:** multiple experiments (0.1 $\rightarrow$ 0.0 $\rightarrow$ 0.1), plus correct parameter grouping.
- **Optimizer:** switch to AdamW and tune β parameters.
- **Dropout:** sweep 0.1, 0.05, 0.0.
- **Block & batch size:** configuration tuning.
- **Residual projection scaling:** GPT-2 style residual scaling.
- **Model size:** extensive sweep across architectures (10+ experiments).
- **MLP ratio:** sweep 3.0, 4.0, 5.0, 6.0.
- **SwiGLU with MLP ratio:** sweep 3.0, 4.0, 5.0, 6.0.
- **Dataset visualization:** exploratory plots and statistics.
- **Tokenizer:** NFKC, whitespace pre-tokenizer, EOS handling, `min_frequency`, special tokens, lowercase, title-aware batching.
- **Attention:** sparse attention, sparse masks, bottleneck projections.

6 Key Changes

We report only the modifications that produced clear, repeatable improvements in optimization dynamics (faster convergence and/or lower validation loss). We omit changes that were only marginally better, had negligible effect, or consistently worsened validation loss.

1. **Optimizer (SGD $\rightarrow$ AdamW):** switching to AdamW (with standard transformer-style β defaults) substantially improved convergence stability and lowered validation loss.
2. **Learning rate & scheduler (cosine annealing $\rightarrow$ warmup + cosine decay):** replacing cosine annealing with a short warmup followed by cosine decay reduced early-training instability while maintaining strong progress later in training.
3. **Dropout sweep:** with only 7 epochs, dropout often slowed convergence and could be net harmful. Empirically, lower dropout (and in some cases zero dropout) performed best in this constrained regime.
4. **GPT-2 style residual projection scaling:** we scaled down the initialization of the residual-producing output projections (attention and MLP output), improving early training stability and enabling deeper stacks to train more reliably.
5. **Model size sweep:** we ran an extensive depth/width sweep to match model capacity to the fixed 7-epoch optimization budget. In this setting, moderately smaller models sometimes outperformed larger ones by converging faster on the ~ 7.7 M-token corpus.
6. **Tokenizer with lowercase normalization:** initially we avoided lowercasing, since case can carry meaning (e.g., “Ask HN” vs. “ask”). However, most tokenizer variations did not materially improve optimization, so we revisited normalization. Lowercasing significantly reduced the effective vocabulary by collapsing case variants (e.g., “Show HN”, “show hn”, “SHOW HN”), yielding a denser training signal per token.
7. **Sparse attention (local window + compression ablations):** we performed controlled ablations to isolate what mattered among full causal attention, sparse/local masks, and compression. Because titles are short, local dependencies dominated next-token prediction; restricting attention to local context improved generalization. A bottleneck projection contributed an additional, largely independent regularization benefit. Our best sparse setting (**sparse-b256**) achieved validation loss 1.107, a modest but consistent improvement over the dense-attention baseline of 1.123; **sparse-b64** achieved 1.108 with lower memory cost, so we selected **sparse-b64** as the final configuration.

7 Experiment Summary

Table 1 summarizes the main ablations.

Table 1: Selected Ablation summary.

Change	Notes	Val. loss
Baseline	provided starter run	1.753258931
Run 1	optimizer	1.536795339
Run 2	lr_lr_schedule	1.325103549
Run 7	dropout	1.307376437
Run 9	residual_projection_scaling	1.290185882
Run 10	model_size_sweep	1.264162560
Run 26	tokenizer_with_lowercase	1.123491384
Run 27	sparse_b64	1.108043355
Run 29	sparse_b256	1.106858772
...		
Best	sparse_b64	1.108043355

8 Best Configuration (Final)

Finally, the best config file can be found in configs/best-0130.yaml.

- **Model (GPT-2 style decoder-only):** block size $L = 64$, vocab size $V = 24,000$, $n_{\text{layer}} = 8$, $n_{\text{head}} = 9$, $d_{\text{model}} = 576$ ($d_{\text{head}} = 64$), MLP ratio = 5.0, activation GELU, dropout = 0.0, residual scaling enabled.
- **Attention (sparse):** local attention context = 16; bottleneck projection with `att_intermediate_dim=288` ($288/9 = 32$ per head).
- **Tokenizer (BPE):** vocab size = 24,000 with lowercase normalizer; whitespace pre-tokenizer; EOS handling: tokenize then append EOS id; `min_frequency=2`; domain special tokens enabled.
- **Optimization:** AdamW with learning rate 3×10^{-4} and weight decay 0.1.
- **Schedule:** warmup + cosine decay with warmup fraction 0.05.
- **Training:** batch size 128 (random batch mode), `evals_per_epoch=3`.

9 Results

9.1 Best Result

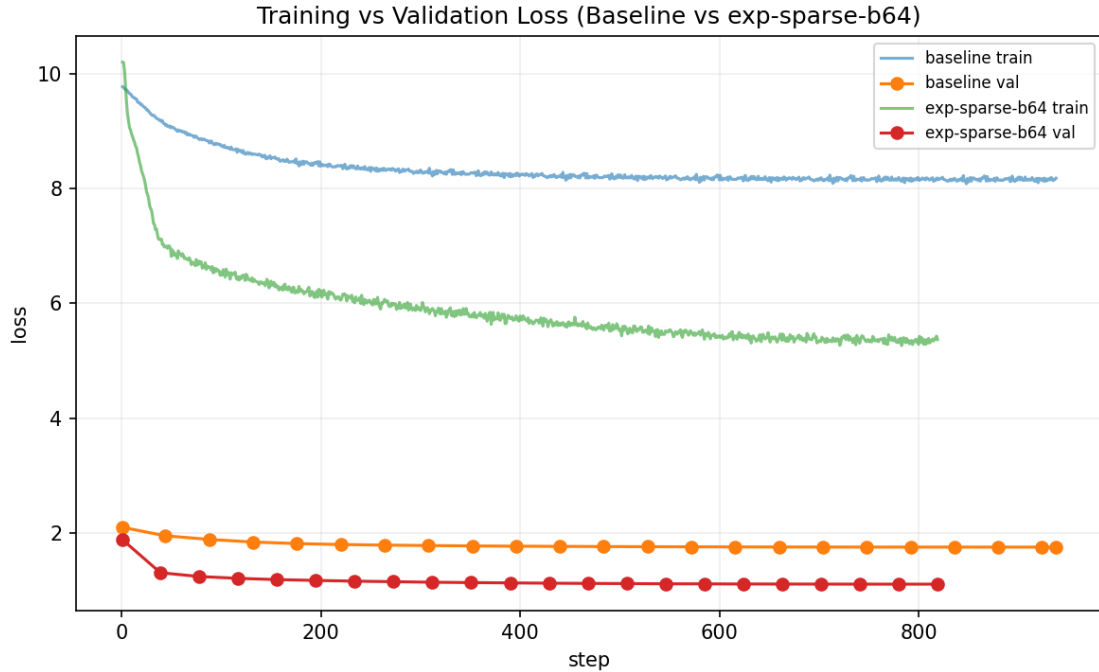


Figure 2: Training/validation loss curves for the baseline and best (sparse_b64) run.

Best validation loss: 1.108043355 (baseline 1.754), achieved at epoch 7.

As shown in Fig 2, train loss steadily decreases, while validation follows closely and only flattens slightly, suggesting no strong overfitting. Overall generalization looks stable with the best run maintaining a lower val curve than baseline..

9.2 Final Contrast Experiments

Figure 3 summarizes the final set of contrast runs, comparing how training and validation loss evolve across the different experiment configurations.

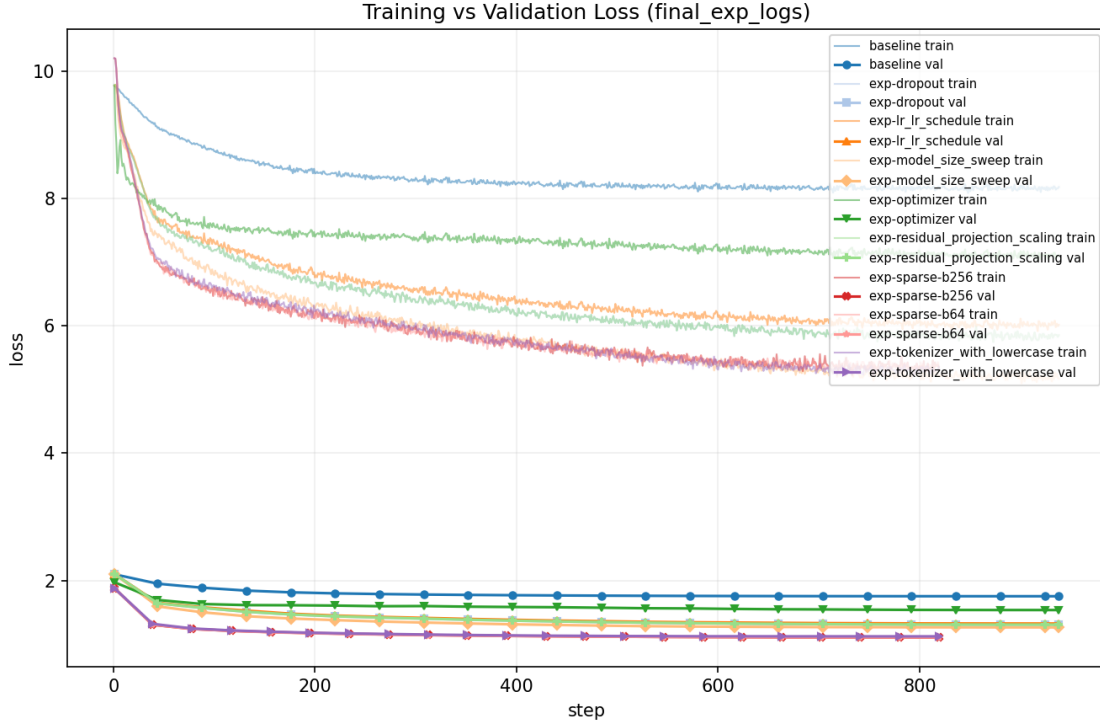


Figure 3: Training/validation loss curves for the final contrast experiments.

10 Reproducibility

- **Command:** “python3 train.py --config configs/best-0130.yaml”.
- **Determinism:** seed, dataset, and split unchanged.

11 Sample Output

After completing this optimization challenge, we also tested the text generation capabilities of both the baseline model and our optimized model using various input prompts. Table 2 presents a comparison of the generated outputs, demonstrating the qualitative improvements achieved through our optimization techniques. Setting: `temp=0.8`, `top_k=50`, `tokens=60`, `samples=3`.

12 Acknowledgements

This work was completed as a co-work/teamwork process with AI assistance.

- Claude and ChatGPT were used for exploration, ideation, and proposing solutions.
- AI coding tools (Claude Code and OpenAI Codex) were used for development and experiments.
- OpenAI Prism was used for writing this report.

References

- [1] Liu Liu, Zheng Qu, Zhaodong Chen, Yufei Ding, and Yuan Xie. Transformer acceleration with dynamic sparse attention. *arXiv preprint arXiv:2110.11299*, 2021.

Table 2: Comparison of text generation outputs between the baseline and optimized models (3 trials per prompt).

Input	Baseline			Optimized (best)		
	Run 1	Run 2	Run 3	Run 1	Run 2	Run 3
AI	AI	AI The:s a	AI	ai for the cloud ?	ai ?	ai is not really the last
OpenAI	OpenAI, and	OpenAI for How with	OpenAI	openai ' s law is looking for a new year for a lot of money	openai – github , the cloud	openai
Maincode	Maincode-	Maincode HN	Maincode	main code review	main code ?	main code
Melbourne Startup	Melbourne Startuping to of and	Melbourne Startup	Melbourne Startup with – HN	melbourne startup	melbourne startup – but in the pandemic	melbourne startup company in a year
Australia Open	Australia Open	Australia Open:	Australia Open	australia open source software tools	australia open source	australia - open source platform